RUHR
UNIVERSITÄT
BOCHUM

RUB

RUHR-UNIVERSITÄT BOCHUM

Performance Overhead of User Plane Integrity Protection in 5G networks

Christian Böhm

Bachelor's Thesis – March 11, 2022.
Chair for System Security.

1st Supervisor: Prof. Dr. Thorsten Holz
2nd Supervisor: Dr. -Ing. David Rupprecht

hgi SYSSEC

Abstract

5G introduces a series of new security features that overcome known issues of the previous mobile generations. One of these features is mandatory integrity protection for user plane data. While this addition protects against, e.g., known-plaintext attacks, it also introduces additional overhead to user plane traffic. In this work, we investigate the computational overhead of different integrity protection algorithms and estimate the additional burden they add to the workload of a device. Our results give an idea of what to expect with mandatory user plane integrity protection and allow for a discussion of the fact that this added security measure remains optional to use.

Eidesstattliche Erklärung

Ich erkläre, dass ich keine Arbeit in gleicher oder ähnlicher Fassung bereits für eine andere Prüfung an der Ruhr-Universität Bochum oder einer anderen Hochschule eingereicht habe.

Ich versichere, dass ich diese Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen benutzt habe. Die Stellen, die anderen Quellen dem Wortlaut oder dem Sinn nach entnommen sind, habe ich unter Angabe der Quellen kenntlich gemacht. Dies gilt sinngemäß auch für verwendete Zeichnungen, Skizzen, bildliche Darstellungen und dergleichen.

Ich versichere auch, dass die von mir eingereichte schriftliche Version mit der digitalen Version übereinstimmt. Ich erkläre mich damit einverstanden, dass die digitale Version dieser Arbeit zwecks Plagiatsprüfung verwendet wird.

Datum	Unterschrift

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Related Work . . . 3
- 1.3 Contributions of this Thesis . . . 4
- 1.4 Outline of this Thesis . . . 5

2 Background **7**
- 2.1 Architecture of 5G . . . 7
  - 2.1.1 Cloudified 5G architecture . . . 8
- 2.2 PDCP Security . . . 9
- 2.3 Integrity Protection . . . 9
  - 2.3.1 Message Authentication Codes . . . 9
  - 2.3.2 The Principle of Message Authentication Codes . . . 10
  - 2.3.3 The Combination of MACs and Encryption . . . 11
- 2.4 Integrity Algorithms of 5G . . . 12
  - 2.4.1 EIA1 . . . 13
  - 2.4.2 EIA2 . . . 14
  - 2.4.3 EIA3 . . . 15
- 2.5 Encryption Algorithms of 5G . . . 16

3 Analysis of Security Algorithms **17**
- 3.1 Experimental Setup . . . 17
- 3.2 Packet and Traffic Properties . . . 18
  - 3.2.1 Keystream Generation vs. MAC Calculation . . . 18
- 3.3 Performance of Integrity Algorithms . . . 20
  - 3.3.1 Poly1305 . . . 20
  - 3.3.2 GHASH . . . 20
  - 3.3.3 Current vs. Proposed Integrity Algorithms . . . 21
- 3.4 Performance of Encryption Algorithms . . . 22
- 3.5 Comparison between Integrity and Encryption . . . 23
- 3.6 Ordering and Combination of Algorithms . . . 24
  - 3.6.1 Parallelized Execution . . . 25
- 3.7 Architectural Impact on Performance Optimization . . . 27
- 3.8 Summary of Findings . . . 29

4 Discussion **31**
4.1 Adaption of New Integrity Algorithms 31
4.1.1 Alternative Encryption Algorithms for One-Time Keys 32
4.2 Requirements by UE and gNodeB 32
4.3 Generality of the Results 33
4.3.1 Limiting Factors 33

5 Conclusion **35**
5.1 Future Work 35

A Acronyms **37**

List of Figures **39**

List of Tables **41**

List of Algorithms **43**

Bibliography **43**

1 Introduction

1.1 Motivation

Mobile networks are ubiquitous in our society [33]. Throughout the years, different mobile generations gradually increased the security and performance of our networks. When switching from our current mobile generation, 4G, to the upcoming 5G technology, a series of fundamental and security-relevant changes is part of the transition process. One actual example of this is the introduction of User Plane Integrity Protection (UP IP) that overcomes critical security flaws in our current 4G networks [48, 49]. To name two prominent attacks that exploit the missing integrity protection in current 4G networks there is the aLTEr attack as well as two IMP4GT attacks.
The lack of integrity protection has the result that an attacker can alter parts of the ciphertext without the recipient noticing. For example, the aLTEr attack makes use of a Man-in-the-middle attacker that changes the destination address of Internet Protocol (IP) packets so that the request is diverted to a Domain Name System (DNS) server under the adversary's control. Then, the mischievous DNS server is able to perform DNS Spoofing. As a result, the domain entered by the victim is now resolved to a malicious IP address and not to the IP address that is actually representing the domain. This may pose a security risk because the user is redirected to a website under adversarial control, rather than visiting the intended site. In a nutshell, the lack of integrity protection enables DNS redirection attacks in 4G [2].
Besides the missing integrity protection, IMP4GT also exploits a reflection mechanism of the IP stack of mobile operating systems. This allows an attacker to build an encryption and decryption oracle and has the overall effect that packets can be decrypted as well as injected by the attacker [4].
With 5G, UP IP becomes *mandatory* to implement.

Although this is a promising step forward, the 3rd Generation Partnership Project (3GPP) only specifies the implementation as mandatory and leaves it *optional* to use for network operators. This leads to a situation where 5G Standalone (SA) equipment must implement full-rate UP IP in 3GPP Release 16. However, operators can still decide whether UP IP is eventually enabled for a specific connection. Integrity protection requires additional steps, which introduce additional overhead [33]. This overhead threatens the high-performance goals of 5G and makes the opt-in less attractive for network operators. As we are in the initial phase of 5G's rollout, we have only minimal knowledge about the *practical* implications of UP IP, the degrees of freedom left for optimization, and how both influences the willingness of operators to enable this optional feature.

Ongoing discussions between operators and vendors indicate that the main concerns regarding UP IP involve latency, throughput, and power consumption [5]. Such performance impairments jeopardize critical sales criteria of 5G, and the existing security threats of missing integrity protection are not transparent to consumers. Besides these strategic assumptions, the actual impact of UP IP remains unclear. Since mobile devices use different baseband chips with fundamental architectural differences, the performance impairments might differ across the various existing implementations. Consequently, we are currently not in a state to compare the performance of different algorithms, their computational load on the existing basebands, and the performance impairments these setups introduce for 5G. The varying hardware setups of base stations complicate this missing evaluation step even further [20].

1.2 Related Work

As the upcoming mobile generation 5G is the first to implement integrity protection for user plane traffic, prior work in this context is limited. However, we can learn from the history of cryptography in previous mobile generations and related work in the context of performance optimizations for algorithms.

Cryptography in Mobile Networks. Encryption has been a feature of mobile networks since the first widely-available generation 2G (GSM). Prior work shows how these algorithms can be broken [12] or even are weakened by design [13]. The next generation 3G (UMTS) introduces integrity protection for control traffic and stronger cryptographic algorithms based on KASUMI, which is vulnerable to related-key attacks [16], and SNOW3G [25], which is still in service today in 4G and 5G. To further improve the security standard, our current mobile generation 4G (LTE) introduces the Advanced Encryption Standard (AES) and ZUC to be used by the encryption and integrity protection algorithms.

Performance Optimization of Algorithms. Over the past years, there has been research on optimization of the cryptographic procedures used, such as for KASUMI [40, 52], SNOW3G [51, 36], ZUC [35], and mainly for AES [37, 10]. There has also been research on the complexity of these procedures [26, 27]. Moreover, the literature studies improvements of the underlying encryption (stream ciphers). While those algorithms are part of the integrity protection algorithm, the additional step for computing a MAC is omitted. Our study compares the existing algorithm with alternatives and discusses optimization strategies independent of the algorithm.

With quite a substantial amount of related work in the fields of cryptography in mobile networks as well as performance optimizations of the algorithms that provide cryptography, the amount of research on UP IP and its performance impact is constrained, and to our knowledge, virtually non-existent. While there is research on the underlying algorithms that provide UP IP, such as SNOW3G, there is a lack of literature on a comparison of all the algorithms that can provide integrity in 5G networks. In addition, there have not been many proposals made for new algorithms to be implemented in the mobile context despite the fact that the ones currently in use are superseded in terms of performance by more recent algorithms, like SNOW-V [20].

1.3 Contributions of this Thesis

Considering 5G is built upon the basis of 4G, the security algorithms are the same as in 4G [3], meaning also 5G is possibly subject to attacks like aLTEr.
Therefore, members of the 3GPP - a committee for mobile telecommunications - tried to emphasize the importance of mandatory integrity protection [1]. However, this proposal was rejected by the vendors due to performance impairments. The latest state of this matter is that the support of user plane integrity protection is mandatory to implement. The use, however, stays optional and is up to the decision of the provider [1].
In my thesis, we investigate the performance overhead of UP IP in 5G networks and propose mechanisms to reduce the performance impact. To achieve this goal, throughput measurements are performed on the algorithms that provide integrity protection and on proposed parallelized versions of these algorithms. These measurements refer to implementations of the official specifications of the algorithms.
Our results indicate significant performance differences within the current state of the art of mobile network integrity protection algorithms. Our experiments demonstrate how conceptual changes like parallelization and pre-computation improve the latency induced by UP IP for both established and novel algorithms. We observe that performance rates depend on hardware accelerations or the underlying instruction set. This implies the potential to optimize performance further through intelligent implementations. Our experiments are the starting point to make UP IP not only a feature mandatory to implement but also *beneficial* to support. By highlighting different ways to reduce the induced performance overhead, we take the first step towards this goal. In our work, we make the following contributions:

- We propose and evaluate context-sensitive and conceptual optimizations of existing algorithms. In particular, we analyze the performance of algorithms in characteristic use case scenarios and assess their potential for parallelization.
- We suggest alternative algorithms for integrity protection and evaluate their performance gains in contrast to the current state.
- To emphasize the potential of certain optimizations, we elaborate on the process to adopt an improved integrity protection setup for future implementations and the 3GPP specification.

1.4 Outline of this Thesis

The essential content of this thesis is divided into three parts. The first part (§2) provides information about the architecture of 5G (§2.1), the background of UP IP (§2.3), as well as the algorithms in use that provide it (§2.4).
Subsequently, the second part (§3) goes into detail about the analysis of the protection algorithms; including an overview of packet and traffic properties (§3.2) along with a theoretical and practical study of the algorithms (§3.3, §3.4). The latter is underlined by measurements that are additionally extended for a parallelized version of the integrity protection algorithms (§3.6). Furthermore, the limitations and opportunities of different architectures are discussed (§3.7), followed by a summary of all findings (§3.8).
Finally, the last chapter examines the adaption, needed to implement the proposed integrity algorithms (§4.1) and specifies the generality of the results of this thesis (§4.3).

2 Background

At the beginning of each connection, the phone and network perform an Authentication and Key Agreement procedure to establish the necessary prerequisites to set up a secure transmission channel. Control data, which manages the connection, can be encrypted and integrity protected from this point. User data, the actual IP data, is encrypted and, since 5G, optionally integrity protected.

2.1 Architecture of 5G

The overall structure of 5G and its components can mainly be broken down into three components. Those being the **User Equipment (UE)**, the 5G **Radio Access Network (RAN)** as well as the 5G **Core Network**.

- UE can be any cellular device that is used by an end-user, such as a mobile phone or a laptop with a mobile broadband adapter. The UE connects to base stations called Next Generation NodeB (gNodeB) that are part of the RAN.
- The RAN is responsible for the link between individual devices and other parts of the network through radio connections, such as the core network. UP IP is applied to the traffic between the RAN and UE to ensure the integrity of the data sent between these two instances.
- The 5G Core Network is the heart of 5G. It ensures secure and reliable connectivity to the network for end users whilst also giving access to its services. The 5G Core Network provides functions for the network, such as authentication and authorization, subscriber data management, among others [23].

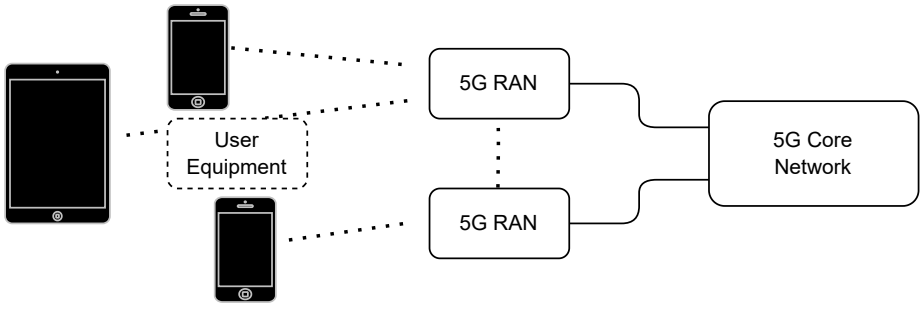


Figure 2.1: RAN Architecture in 5G

2.1.1 Cloudified 5G architecture

One big change coming with the evolution of the 5G architecture is the possibility of cloudifying parts of the architecture. This allows for different network functionalities to be virtualized in the cloud [6]. To enable virtualization, hypervisors are used to run virtual machines. While older cellular networks required dedicated hardware to provide different functionalities, the option of cloudifying parts of the architecture results in more general-purpose hardware being used. This in turn leads to a reduction in equipment and deployment cost [53].
Additionally, it is possible to cloudify parts of the access network. The resulting architecture is known as Cloud Radio Access Network (C-RAN) (cf. Figure 2.2). To differentiate which functions are suitable to be moved into the cloud, one has to consider latency. Some functions, including access network scheduling or modulation, require real-time computation; therefore, they are not processed in the cloud. Other non-real-time functions, however, are well suited for cloud computation in software environments. Those include, e.g., user-plane encryption and integrity protection [17].

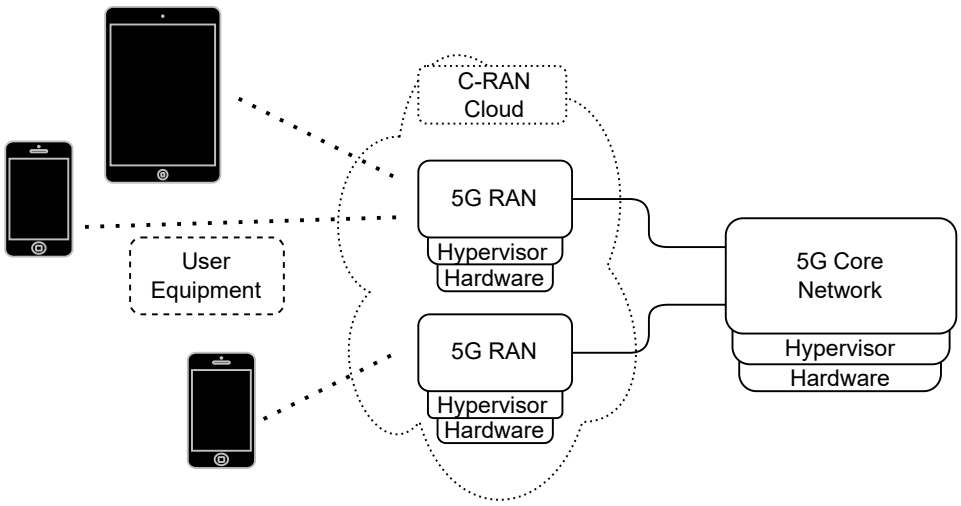


Figure 2.2: C-RAN Architecture in 5G

2.2 PDCP Security

The Packet Data Convergence Protocol (PDCP) layer is responsible for the encryption and integrity protection and has a maximum transfer unit of 8188 bytes (excluding jumbo frames). Discussions with the industry revealed the fact that the length of packets to be integrity protected is typically smaller than 1200 bytes. Further, user traffic is divided into bearers that fulfill specific quality requirements. Usually, these consist of the default bearer for Internet service, and two bearers for Voice over LTE (VoLTE) traffic. Bearers can also have different security parameters such as algorithms [7]. Furthermore, the PDCP uses a MAC-then-Encrypt scheme [8].
Moreover, the PDCP layer is likely to be moved to the cloud using software implementations without hardware acceleration, according to ETSI Security Algorithms Group of Experts (SAGE) [34].

2.3 Integrity Protection

In the context of cryptography, message integrity is the validity of a transmitted message. This means that a message that has integrity has not been altered during transmission. The goal of integrity is achieved by the use of a Message Authentication Code (MAC) - it indicates whether or not a message has been subject to change during transmission.

2.3.1 Message Authentication Codes

A MAC, also known as tag or cryptographic checksum, is the output of a MAC algorithm. This output is used not only to provide a way of ensuring the integrity of a message but also of ascertaining the authenticity; i.e., the certainty about the origin of a message [43]. As its two input parameters, a MAC algorithm takes a message that is to be protected and a secret key that is known to both the sending and receiving parties. MAC algorithms are based on block ciphers or hash functions to calculate the tag, which itself can only be calculated by parties having access to the secret key [43]. Such a tag has a fixed length and is independent of the length of the input message.

2.3.2 The Principle of Message Authentication Codes

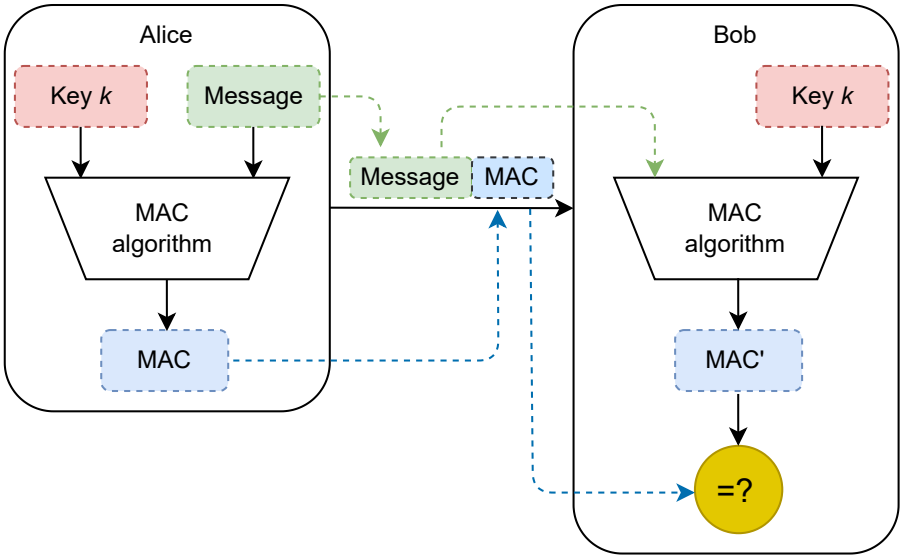


Figure 2.3: MAC Principle

Shown in Figure 2.3 are two parties; Alice and Bob that want to communicate with each other. In this case, Alice on the left sends a message to Bob, who receives the message on the right. If Alice wants to send an integrity-protected message to Bob, she calculates the MAC of the message with the secret key k and attaches the tag to the message before sending the packet. When Bob receives the packet, he also calculates the MAC' of the message that he received by using the same secret key k. Finally, he compares the tag of his calculation with the tag, sent by Alice. If both tags are identical, the message has not been tampered with during transmission.

2.3.3 The Combination of MACs and Encryption

While integrity and authenticity are secured by the use of MACs, another traditional goal, i.e., confidentiality requires the use of encryption. To ensure all three security goals simultaneously, encryption and integrity protection can be combined to form an Authenticated Encryption Scheme. To achieve this, there are mainly three possible composition methods of bringing encryption and integrity protection algorithms together [14]:

- **Encrypt-and-MAC**
  The message is encrypted and the MAC is calculated over the message in plaintext. The MAC is appended to the ciphertext.
- **MAC-then-Encrypt**
  The MAC is calculated over the message in plaintext. Then, the MAC is appended to the message before the concatenation of both is encrypted.
- **Encrypt-then-MAC**
  First the message is encrypted. Subsequently, the MAC of the ciphertext is calculated which is then appended to the ciphertext.

2.4 Integrity Algorithms of 5G

5G and LTE share the same set of security algorithms. While the algorithms are the same, their naming differs. Throughout the thesis, we refer to the Long Term Evolution (LTE) nomenclature where the integrity algorithms are called EIAx (Enhanced Integrity Algorithm) wherein both cases $x = 1, 2, 3$. Our main focus is on the integrity procedures EIA1, EIA2, and EIA3. Only EIA1 and EIA2 must be implemented by both the user and the network [9], while EIA3 is optional. All algorithms share the same basic interface and use individual underlying base algorithms. As input, all algorithms require a 128-bit Integrity Key (IK), a 32-bit count, a 5-bit bearer identity, a 1-bit direction of the transmission, a 32-bit length of the message, and the message itself; as output, they produce a 32-bit MAC [24]. All algorithms do have a linear time complexity and either a constant or linear space complexity [26, 27]. Currently, the following integrity protection algorithms with the underlying algorithm are specified:

- EIA1: SNOW3G
- EIA2: AES (CMAC mode)
- EIA3: ZUC

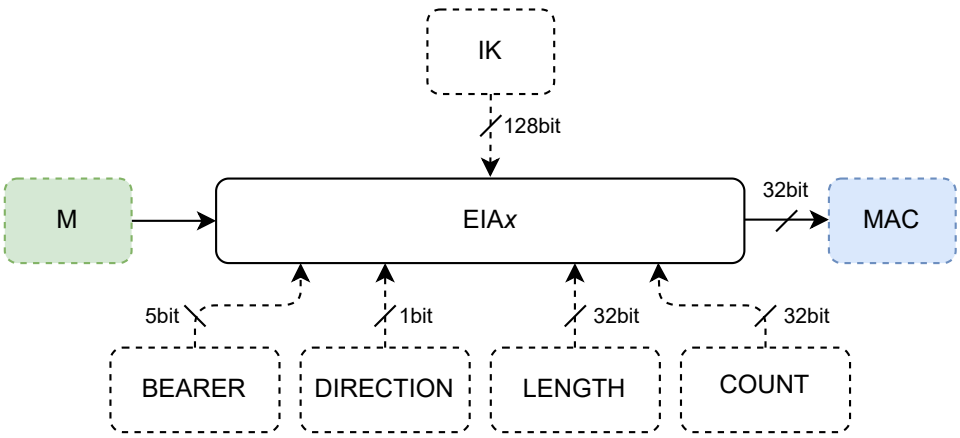


Figure 2.4: Inputs and Output of EIAx

2.4.1 EIA1

EIA1 uses the stream cipher SNOW3G and was originally specified for use in 3G systems in 2006 [25] under the name UEA2. The message input of maximum length 2^{32} bits is divided into 64-bit chunks. SNOW3G is used to compute a keystream of five 32-bit words (z_1,z_2,...,z_5). This keystream is used to calculate the MAC, making use of two mapping functions (EVAL_M, MUL) in $GF(2^{64})$.

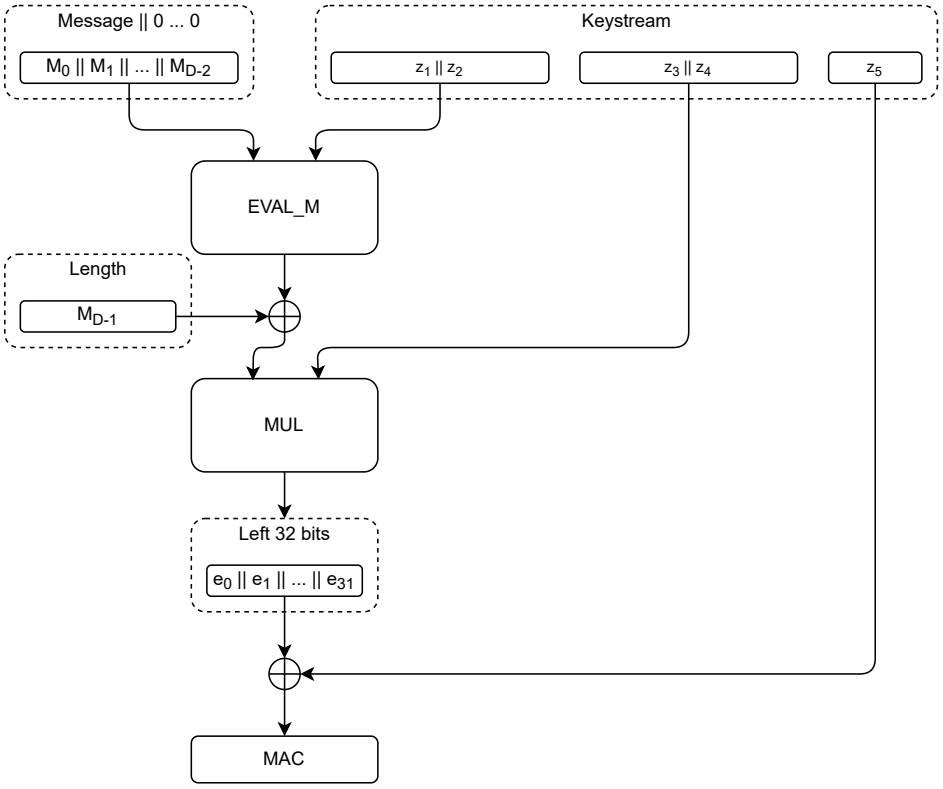


Figure 2.5: Structure of EIA1's MAC Calculation

The function **EVAL_M** corresponds to the evaluation of a polynomial at a secret point. The bits and length of the message define a polynomial $M \in GF(2^{64})$. The resulting polynomial is then evaluated at the point $P \in GF(2^{64})$ which is characterized by $z_1 \,||\, z_2$.

MUL, on the other hand, resembles a multiplication of the 128-bit long output of EVAL_M XORed with *Length*, and $Q = z_3 \,||\, z_4$ in $GF(2^{64})$. Thereby it maps 192 bits to 64 bits [25].

2.4.2 EIA2

EIA2 makes use of AES in CMAC mode to calculate the MAC [19]. The sequence number, bearer identity, and direction are appended to the message and serve as input for the AES-CMAC algorithm. If the message can be split into blocks of 128 bits without remaining bits, the subkey K_1 is XORed onto the last message block. Else, i.e., the message is not a multiple of 128 bits, the last block is padded with a single '1' and as much '0' as are needed to reach 128-bit block size. Subsequently, K_2 is XORed onto the last message block (cf. Figure 2.6).

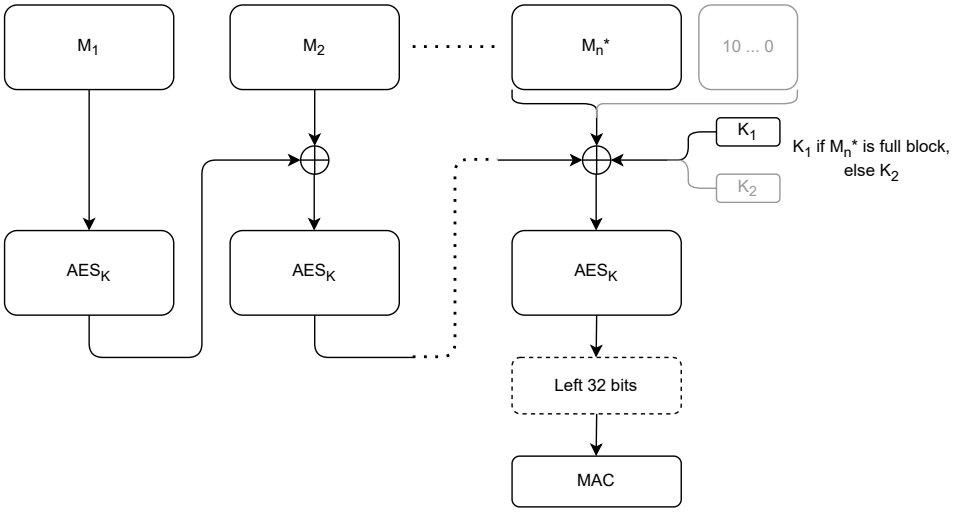


Figure 2.6: Structure of EIA2's MAC Calculation

Algorithm 1 Calculation of Subkeys K1, K2

$R_{128} = 0^{120}10000111, b = 128, L = AES_k(0^b)$
if $MSB_1(L) = 0$ **then**
    $K1 = L << 1$
else
    $K1 = (L << 1) \oplus R_b$
end if
if $MSB_1(K1) = 0$ **then**
    $K2 = K1 << 1$
else
    $K2 = (K1 << 1) \oplus R_b$
end if
return $K1, K2$

2.4.3 EIA3

EIA3 is the second algorithm to utilize a keystream for building the MAC and is based on the stream cipher ZUC. ZUC generates a keystream that is 64 bits longer than the input message of length 'LENGTH' bits. The MAC is then calculated via XOR operations. A sliding window uses parts of the keystream z to be XORed if a bit of the message is set (cf. Algorithm 2).

Algorithm 2 MAC Calculation of EIA3

1: $T = 0^{32}$
2: **for** $i = 0, 1, 2, ..., LENGTH - 1$ **do**
3: **if** $M[i] = 1$ **then**
4: $T = T \oplus z_i$
5: **end if**
6: **end for**
7: $T = T \oplus z_{LENGTH}$
8: return $MAC = T \oplus z_{32 \cdot (L-1)}$

2.5 Encryption Algorithms of 5G

We refer to the same nomenclature for the encryption algorithms as for the integrity algorithms. Thus, we name these algorithms EEAx (Enhanced Encryption Algorithm). The encryption and integrity algorithms also share the same input with the only difference being another key: EEAx uses a Confidentiality Key (CK) instead of an IK as present in EIAx. Moreover, the underlying base algorithms are identical to EIAx, however, with a different mode of operation for AES in EEA2:

- EEA1: SNOW3G
- EEA2: AES (CTR mode)
- EEA3: ZUC

For the sending direction, the encryption algorithm generates a keystream block which is then XORed with the message, resulting in the ciphertext block (cf. Figure 2.7). Because of the symmetric nature of this scheme, the receiving party substitutes the plaintext M with the ciphertext block and calculates the plaintext M by XORing the same keystream block with the ciphertext block [24].

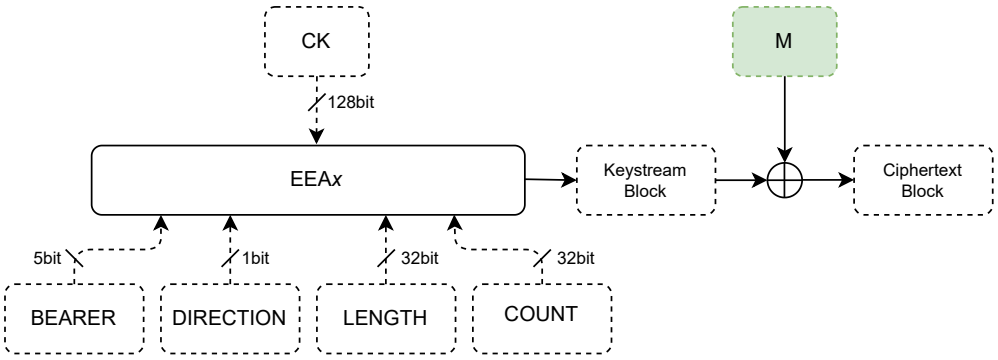


Figure 2.7: Inputs and Output of EEAx

3 Analysis of Security Algorithms

To get a better understanding of the expected overhead, we analyze the performance of security algorithms in the presence of different influencing factors (cf. Figure 3.1). We first document the experimental setup of our reference measurements and then begin our evaluation with the packet and traffic properties (§3.2). In the next step, we focus on the different possible choices of algorithms (§3.3) and concepts to order and combine them (§3.6). Finally, we take the influence of different architectures into account (§3.7).

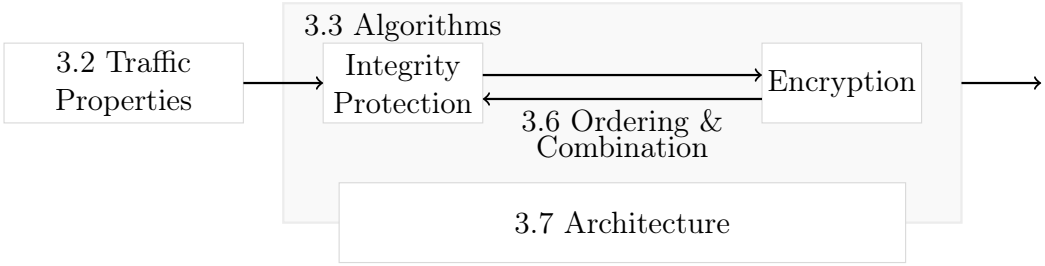


Figure 3.1: System Concept of Influencing Factors.

3.1 Experimental Setup

For all experiments, we use existing C/C++ implementations of the different EIA/EEA algorithms as implemented in the srsRAN suite [28], which refer to the official specification of EIA1, EEA1, EIA3, and EEA3. For the AES-based algorithms (EIA2/EEA2), srsRAN uses the `mbedtls` library based on the underlying Intel intrinsic. Where applicable for our different evaluation goals, we extend this selection of implementations by alternative algorithms.

We perform all experiments on an AMD Ryzen 7 PRO 4750U. Each experiment runs on a bounded core, if not stated otherwise. We measure the runtime by calculating the difference between start and end time. For the throughput, we run the algorithm for 5 seconds and measure the total amount of data that was integrity protected. We vary the input length throughout the experiments to show the effects of different sizes on runtime and throughput.

3.2 Packet and Traffic Properties

The characteristics of mobile traffic highly depend on the scenario, e.g., web traffic can have bursts, whereas VoLTE traffic has a more constant rate of small packets. By adjusting integrity algorithms to these characteristics, it is possible to improve their efficiency. Notably, the delay added when a packet is integrity protected, e.g., the *latency* can benefit from this. For example, pre-calculations accomplish parts of the work *before* a packet is available, e.g., by already preparing the keystream and storing it. The separation can result in lower *latency*. However, it might come at the cost of storage space for the pre-computed keystream. Due to their internal structure, only EIA1 and EIA3 can be split into a pre-calculation and a main phase. Furthermore, one requirement for EIA3 to pre-calculate the keystream is to know the packet size in advance because EIA3 calculates a keystream that is 8 bytes bigger than the package that is to be integrity protected. On the other hand, EIA1's keystream does not depend on the packet size because it always produces a stream of 20 bytes independent of the input size. Another obstacle of pre-computation is that one has to know the IV values in advance.

3.2.1 Keystream Generation vs. MAC Calculation

We perform measurements on both the keystream generation phase and the later MAC computation phase to estimate whether a separation of both phases decreases the latency (cf. Figure 3.2). Further, we measure the memory usage for storing the pre-calculated keystream to evaluate the trade-off between latency decrease and memory usage.

EIA1 has a constant keystream generation time, and its MAC computation time is linear to the input size. In contrast, both phases of EIA3 are linear to the input length. Overall, the relative performance of EIA3 is better than EIA1. EIA1 uses a constant of 20 bytes to store the keystream in terms of memory occupation. At the same time, EIA3 uses 8 bytes more than the input length, resulting in linear memory usage.

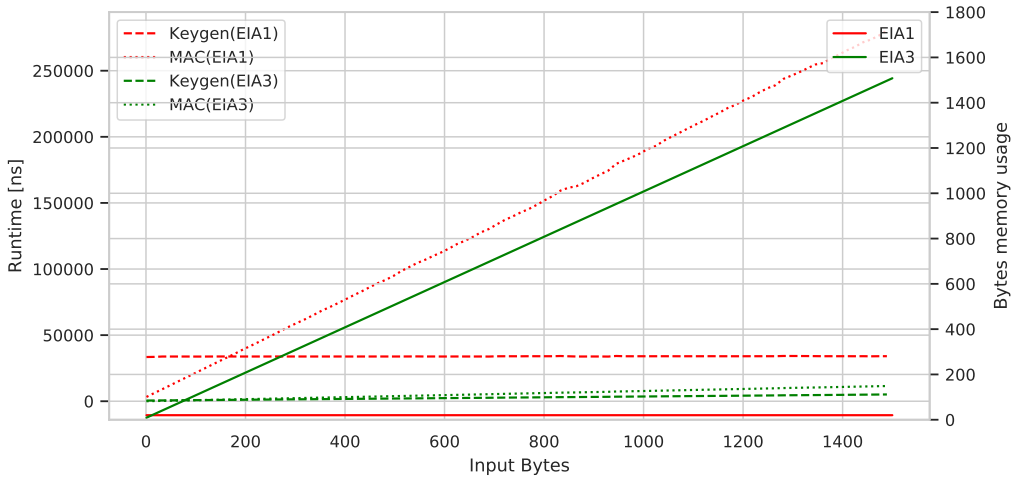


Figure 3.2: Key Stream Generation vs. MAC Calculation vs. Memory Usage

Our experiments indicate that a pre-calculation of the keystream *always* contributes to an improved latency. However, the availability of these improvements and the individual gain depend on the circumstances.

In the case of EIA1, the keystream phase has a constant overhead. For smaller packets, this results in higher *relative* overhead. Consequently, the relative gain is emphasized for smaller packets (below 200 bytes). While this relative gain decreases for bigger packets, the pre-calculation step remains overall beneficial.

In the case of EIA3, the MAC phase depends on the keystream, which directly relates to the size of a packet. Due to this constraint, we still benefit from pre-calculations. However, the packet size must be known in advance. Over-estimating packets is a workaround for this case but can be a drawback for memory usage. In a worst-case scenario, the baseband must store 8300 keystream bytes.

VoLTE traffic is a use case in which both algorithms benefit from pre-calculations. For EIA1, the small packet sizes lead to a high relative performance gain. For EIA3, we can predict the packet sizes and optimize the memory overhead within the baseband.

3.3 Performance of Integrity Algorithms

According to the specification, our current set of integrity protection algorithms involves two different approaches: EIA2 uses AES in Block cipher-based MAC (CMAC) mode; EIA1 and EIA3 use stream ciphers designed explicitly for the context of mobile networks. While stream ciphers can provide adequate performance for encryption, using them for integrity protection may not be the most efficient choice. For example, the MAC computation in EIA2 and EIA3 requires an entire run of the underlying encryption algorithm.

Opposed to the EIA algorithms of the mobile domain, other integrity protection concepts provide better optimizations and could introduce performance improvements for mobile networks. Examples of such algorithms are GHASH or Poly1305. We compare the throughput of the existing algorithms and the suggested alternative algorithms.

3.3.1 Poly1305

Poly1305 is a MAC algorithm that is said to deliver consistent high speed for many CPUs whilst also providing the ability to take advantage of additional hardware to reduce latency [15]. Poly1305 can work in conjunction with a cipher, e.g., AES or the stream cipher ChaCha to form an Authenticated Encryption with Associated Data (AEAD) scheme [32].

3.3.2 GHASH

GHASH is part of the Galois/Counter Mode (GCM), one mode of operation recommended by the National Institute of Standards and Technology (NIST) [18]. The GHASH function is responsible for the authentication part, thus being the integrity algorithm of the GCM. To authenticate a message, the GHASH function makes use of universal hashing based on multiplication in the binary field $\mathbb{F}_{2^{128}}$ [39].

3.3.3 Current vs. Proposed Integrity Algorithms

Figure 3.3 summarizes the throughput performance of the three main integrity algorithms (EIAx) in comparison with a hardware-accelerated version of EIA2 (HW), Poly1305, and the GHASH function as part of GCM. In our experiments, we measure the throughput for each input length for five seconds. We repeat the measurements six times and aggregate the results by reporting the average throughput across all repetitions. Our results indicate that the group of conventional algorithms that are present in 4G and 5G represents the lower spectrum of performance, with EIA2 leading in performance among those three algorithms. The hardware-accelerated version of EIA2 performs even better and reaches around 3.4 times the throughput of the plain software implementation of EIA2. As far as the proposed algorithms go, GHASH has a rather poor performance, similar to that of EIA1 which is the slowest of the bunch. In contrast, Poly1305 provides the best performance within the test set by having at least twice the throughput of every other tested algorithm.

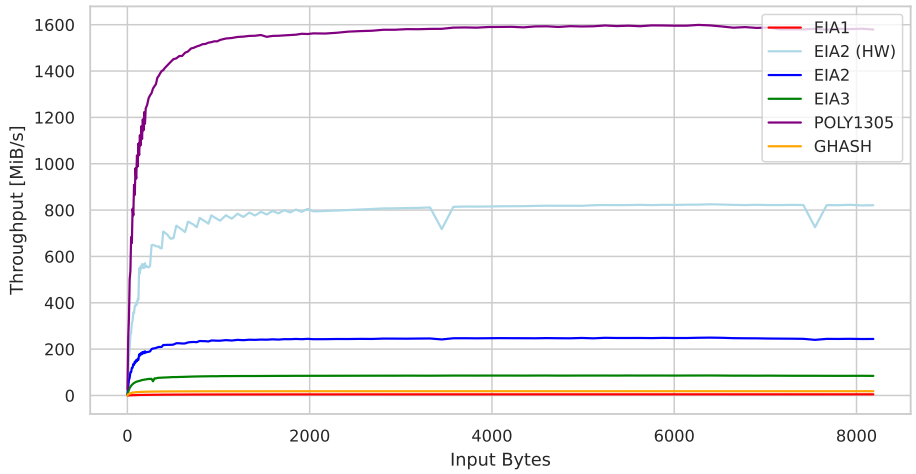


Figure 3.3: Throughput of Existing and Alternative Integrity Protection Algorithms

Our results indicate apparent relative differences between the algorithms under test. Although the absolute performance only represents our test architecture, we expect the relative throughput differences to remain relevant in specialized implementations and on realistic hardware settings. For UP IP in 5G, this implies that we need to identify the best-performing algorithms for each architecture, as otherwise, we will have an unneeded performance overhead. Based on our results, we suggest using Poly1305 as a performant alternative to the current set of specified UP IP algorithms.

3.4 Performance of Encryption Algorithms

Figure 3.4 depicts the throughput performance of the encryption algorithms (EEAx). Just as with the integrity algorithm measurements, we measure six times for five seconds for each input size, but this time only over an input size of 1 to a maximum of 1500 bytes. Unsurprisingly, the order of algorithms for the encryption algorithms is identical to that for the integrity algorithms - that is, EEA2 (HW), which is based on AES, performs best, followed by EEA3 and EEA1 at last.

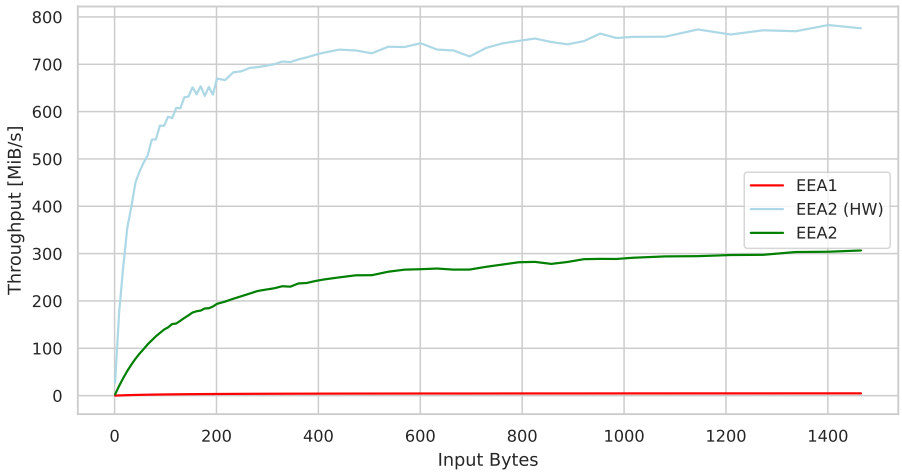


Figure 3.4: Throughput of Encryption Algorithms

3.5 Comparison between Integrity and Encryption

Looking at the relation of each integrity and encryption algorithm at an input length of 1465 bytes, it becomes distinct that EIA2(HW) and EEA2(HW) are roughly performing equally well, with both reaching throughputs of ∼775 MiB/s and ∼735 MiB/s, respectively. Conversely, EEA3 approximately has 3.7 times as much throughput as EIA3 despite both being based on the same underlying algorithm, namely ZUC. EEA1 and EIA1 are relatively alike in terms of their throughput with EEA1 achieving a slightly higher throughput of ∼4.66 MiB/s compared to ∼4.41 MiB/s of EIA1.

Table 3.1: Absolute Throughput of EIAx and EEAx for 1465 bytes input

Integrity	Throughput [MiB/s]	Encryption	Throughput [MiB/s]
EIA1	4.41	EEA1	4.66
EIA2	234.86		
EIA2(HW)	775.39	EEA2(HW)	735.03
EIA3	82.03	EEA3	301.76

3.6 Ordering and Combination of Algorithms

One conceptual option to improve the performance of integrity protection is the parallelization of ciphertext and MAC computations. To this end, MAC-then-Encrypt schemes require a MAC of the message before applying the ciphering. However, we are still able to perform the computation *simultaneously*. The approach to this is as follows: First, we encrypt and compute the MAC independently, which can be done as both algorithms access the plaintext (cf. Figure 3.5). In contrast to the classical MAC-then-Encrypt sequence, we do not append the MAC to the plaintext before the entire packet is encrypted. Instead, the encryption algorithm puts out additional keystream bytes next to the encrypted message. Those keystream bytes have the same length as the MAC, i.e., 4 bytes. In a joined step, we encrypt (XOR) the MAC with the keystream bytes and append the encrypted MAC to the encrypted packet.

The advantage of this approach is the parallel execution, which potentially means a decrease in latency and an increase in throughput. Nevertheless, the performance is limited by the slowest of both algorithms. Further, multithreading can introduce an additional overhead, exceeding the performance gain by the parallel execution. For an optimal implementation without threading overhead, the best-case performance gain can be computed as follows: $gain = min(EIA, EEA)/EIA_EEA$. EIA_EEA is the throughput of the single-threaded concatenation of EIA with EEA. EIA and EEA are their throughput values separately. With this ideal value, we analyze whether and which conditions must be fulfilled for multithreading to be worthwhile.

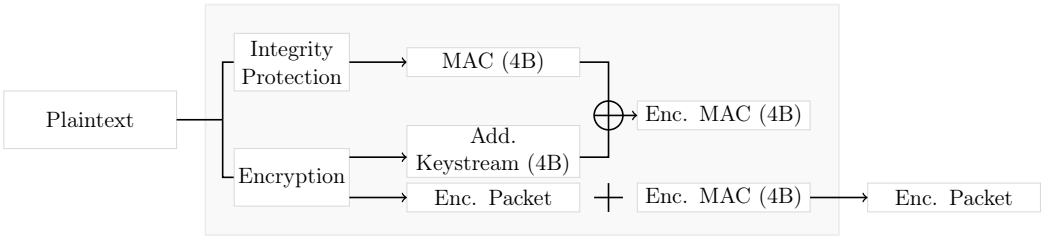


Figure 3.5: Schematic Representation of the Parallel Execution for TX Direction.

3.6.1 Parallelized Execution

We implement a parallelized and multi-threaded version that uses three worker threads. One thread deals with the MAC calculation, while simultaneously, a second performs the encryption. Subsequently, the MAC is encrypted and appended to the message in a final thread. All threads communicate via a thread-safe queue. For our measurements, we limit ourselves to the combination of three algorithms, e.g., EIA1 goes with EEA1 to form EXA1. We compare the throughput of the parallelized versions with the single-threaded versions and provide a factor that shows the performance gain depending on the input length (cf. Figure 3.6). This factor is calculated by dividing the throughput of the multithreaded program by the throughput of the single-threaded program.

The most significant advantage of utilizing a multithreading implementation is visible for EXA1. Its performance gain of 1.45x is available for the entire set of different input lengths in our setup. In contrast, a similar performance gain for the multi-threaded version of EXA2 only becomes available for at least ∼800 bytes input length. We observe similar behavior for EXA3, where a minimum input length of ∼300 bytes is necessary for the same performance gain. Our measurements indicate that combinations of slower algorithms are better suited for multi-threading with smaller input sizes, where the threading overhead becomes less noticeable.

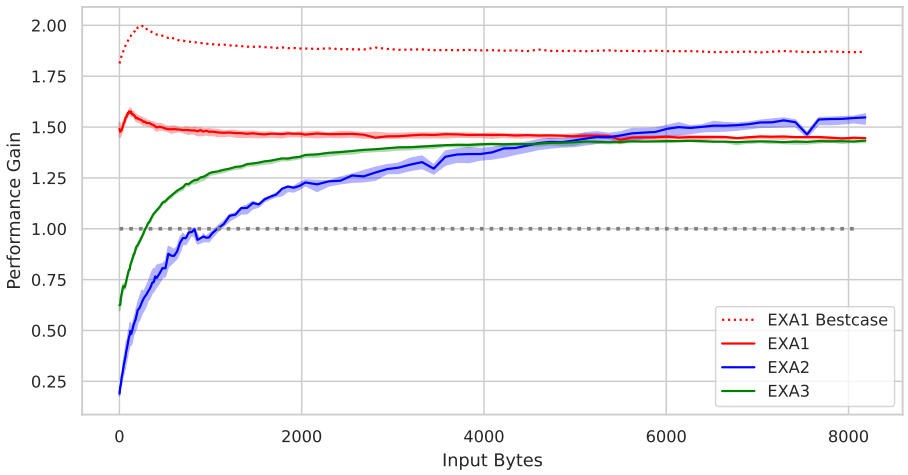


Figure 3.6: Performance Gain of our Multi-Threaded Integrity Computation over Single-Threaded variant.

When comparing our implementation with the best-case performance values of EXA1, we see a fixed offset of factor 0.4. This indicates that our implementation differs from the perfect implementation by a factor of 0.4. One possible reason for this is the threading overhead or the blocking queue communication.

Parallelization increases the throughput and thereby has a positive effect on the latency. At the same time, it depends on additional resources like multiple cores for multi-threading. Our parallelized implementation is a baseline implementation that is open for further improvements. Many optimization factors can keep the threading overhead as low as possible and make the multithreading approach worth using. Nevertheless, the limiting parallelization factor is the slowest algorithm, emphasizing the need for performance-optimized algorithms.

While we demonstrate the performance gain with two main threads, this approach can be extended to multiple threads (cores). This depends on the use case and available resources. An example of the high potential of this optimization is base stations, where massive multi-core implementations provide powerful resources.

The structure of MAC-then-Encrypt allows that the performance optimization through parallelization can be obtained in the transmitting (TX) direction and not in the receiving direction. Considering possible traffic asymmetries within the mobile network makes it worth reevaluating the MAC-then-Encrypt scheme. For example, having a large amount of downlink traffic allows a performance optimization through the parallelization at the base station. However, as the phone receives, it needs to encrypt and check the MAC sequentially, throttling the average system's performance.

3.7 Architectural Impact on Performance Optimization

An important factor for the performance of security algorithms is optimizing the processor's features or directly integrating them into hardware. While a full-fledged performance comparison of different architectures and hardware implementations is out of scope for this work, we provide a preliminary analysis of potential enhancements. We focus on the AES-based EIA2 algorithm and compare a pure software version with an implementation that uses the underlying x86 AES instructions.

In our measurements, we compare the different variants of implementations and how using the underlying instruction set can affect the overall performance. Figure 3.3 includes the throughput of EIA2 (HW) in contrast to a pure software implementation of EIA2. Our experiments indicate how using the x86 instruction set increases the performance of the algorithm by roughly four times compared to the software version (EIA2).

Our experiments underline the assumption that hardware optimizations increase the overall performance of an algorithm. As this complicates the implementation process, it is reasonable to ask how vendors use hardware optimization. The market covers many specialized architectures on one side. On the other side, we have specialized integrity algorithms (EIA1, EIA3) and one (EIA2) based on the well-established AES cipher. Combining both sides is not trivial and has several implications for real-world implementations. To get a better understanding of these implications, we analyze the current market situation.

We differentiate between the two kinds of hardware optimization. The first possibility is to implement the algorithm directly in hardware. The second is to use Single Instruction Multiple Data (SIMD) instructions, allowing performant and parallelized implementations without specialized hardware. We analyze four major baseband vendors, each building on top of their architecture. We also include the x86 architecture, as we expect base stations to utilize the architecture. The results are shown in Table 3.2.

Table 3.2: Vendors and ISA Support for Crypto Algorithms

Vendor	Architecture	Crypto ISA	SIMD ISA	Refs.
Qualcomm	Hexagon DSP	-	HVX	[46]
Mediatek	MIPS (since 2018)	-	DSP extension	[31, 41]
Samsung	ARM	AES, PMULL	Neon	[29, 11]
Huawei[1]	N/A	N/A	N/A	
Intel	x86	AES, GCM	AVX	[22, 30]

Two architectures support the direct hardware acceleration for AES and GCM (ARM, x86). Only the AES acceleration is of interest for mobile networks used for EXA2. Regardless, the GCM acceleration could also become relevant for mobile networks if the 3GPP wants to add GCM as an integrity algorithm. Similarly, the baseband vendors can implement hardware accelerators for the current suite of algorithms (EIA1/EIA3). This would increase the performance but comes at the cost of chip space.

Four architectures support SIMD instructions, which help implement the integrity algorithm in software. Related work proposes implementations for Poly1305, and GCM with SIMD instructions [44, 50]. Future work needs to improve the existing integrity algorithm for SIMD instructions. Alternatively, the industry could adopt the already optimized algorithm in the standard.

[1] Unfortunately, we could not find public information about the Huawei HiSilicon Balong baseband.

3.8 Summary of Findings

Table 5.1 summarizes our findings for the integrity algorithms in a relative manner. To compare the throughputs of the algorithms, the throughput at maximum input length (8188 bytes) is considered for each algorithm. The second column displays the relative throughput of the integrity algorithms compared to the fastest, i.e., Poly1305. Column three presents the performance gain of hardware acceleration for EIA2. The fourth column shows the performance gain of parallelized executions of EXA1,2,3. The last column demonstrates the best possible gain of parallelization that would theoretically be possible when not accounting for threading overhead or other factors.

Table 3.3: Summary of Findings

Algorithm	Performance (Throughput)	HW acceleration	Parallelization	Parallelization (best case)
EIA1 / EXA1	0.31%	-	1.45x	1.86x
EIA2 / EXA2	15.47%	1.00x	1.54x	1.95x
EIA2 (HW)	51.54%	3.33x	-	-
EIA3 / EXA3	5.37%	-	1.43x	1.32x [2]
GHASH	1.15%	-	-	-
Poly1305	100.00%	-	-	-

[2] See §4.3.1 for explanation

4 Discussion

Enabling UP IP does introduce overhead for 5G connections. However, our experimental evaluation indicates that specific conceptual changes, alternative algorithms, and optimized implementations can limit this overhead. This helps to maintain the performance goals of 5G and makes this optional feature a realistic choice for productive networks. To support this process, we discuss the necessary steps to consider new algorithms in the specification and the requirements for their implementation. In a final step, we point out helpful directions for future work.

4.1 Adaption of New Integrity Algorithms

Our experiments indicate that Poly1305 outperforms other integrity protection algorithms, and introducing it to the mobile context holds potential performance gains. However, the raw algorithm does not fit into the specification's current concept of interfaces (inputs). To overcome this, two different strategies are possible.

One option is to take a step back from the current concept to create a closer connection between the encryption and integrity algorithms and their interaction. This step is in line with the AEAD approach that combines suited couples of algorithms, e.g., AES-GCM, ChaCha-Poly1305. A positive side effect of this is the increased trustworthiness of the combined algorithm. For example, we can assume that ChaCha-Poly1305 is fully secure if the underlying ChaCha encryption is secure [15]. Despite the advantages on different levels, this adjustment requires a more significant conceptual change in the current specification.

The second option is to *adjust* new candidate algorithms to the current concept of individual treatment. Integrating Poly1305 into this setting requires generating a one-time key used as input for the integrity algorithm. We can use an encryption algorithm to accomplish this step, which is comparable to the concept of EIA1. Due to its similarity to an existing algorithm, it is trivial to repeat this step for Poly1305. To realize this for the specification, the 3GPP must agree on an encryption algorithm that provides the one-time key. Options range from the existing SNOW3G, AES, and ZUC algorithms to a new one like SNOW-V or ChaCha.

4.1.1 Alternative Encryption Algorithms for One-Time Keys

SNOW-V is a recently proposed stream cipher developed at Ericsson. It is the evolution of the SNOW3G algorithm which is used since 3G. SNOW-V is aimed at providing high-speed encryption using the advantages of modern systems, such as applying both existing acceleration instructions for the AES encryption round function in addition to the ability of modern CPUs to handle large vectors of integers. In terms of performance, a single-threaded software implementation of SNOW-V is faster than AES in all comparable modes of operation for messages longer than 256 bytes [20]. SNOW-V uses a 256-bit key, which is the same length as the output of the key derivation functions in LTE and 5G.

SNOW-Vi was introduced in 2021 to increase performance even further. It reaches 50% more throughput than SNOW-V in the same software environment with the same 256-bit security level as SNOW-V [21].

ChaCha is a stream cipher developed by D.J. Bernstein in 2008. It is three times as fast as AES in software-only implementations, thus making it a great choice for platforms without specialized hardware [42]. Additionally, it is robust against Side Channel Analysis (SCA), such as timing attacks [38, 42]. Like SNOW-V, it uses a 256-bit key.

4.2 Requirements by UE and gNodeB

While performance is primarily an issue for UE, such as mobile devices, it also impacts the operator's infrastructure, like the gNodeB. Additionally, network components are less resource-constrained than mobile devices and are increasingly often virtualized as well as cloudified [47, 53]. Due to the large and ever-increasing volume of mobile traffic, any performance impairments have a global impact on the resource requirements with further environmental effects due to the power consumption [45]. Introducing efficient alternatives, both through novel algorithms and novel implementation concepts, contributes to the global scalability of this problem statement.

4.3 Generality of the Results

Certain characteristics of our experimental setup limit the scope of our findings. For example, we document performance features in a generic x86 environment with the original C implementations as defined in the 3GPP specifications and not on the architectures, that are present in UE. While these first steps provide us with a general overview of algorithms and concepts, they can only indicate the relative differences between different options. Despite that, these results on the x86 architecture are valuable and especially interesting for the RAN, seeing that the hardware in cloudified systems is general-purpose CPUs and memory [53]. Moreover, the fact that the PDCP layer is likely to be moved into the cloud makes these results even more valuable [34].

4.3.1 Limiting Factors

One limiting factor of the results is the implementations that we use for our measurements. Specifically the performance of these implementations and the question of how comparable the implementations that we measure are with those that would actually be used in practice. For example, our throughput results indicate that GHASH is much slower than Poly1305 (§3.3.3). However, this should not be the case as GHASH is measured to be twice as fast as Poly1305 [20]. This can be explained by a slow implementation of GHASH that does not make use of, e.g., look-up tables.

A further constraint coming with implementation is the problem of comparability between the programs used to create measurements. For example, in §3.3 we measure a performance gain for EXA3 that is greater than the theoretical maximum[1]. The most likely explanation is the fact that our program used for measuring the single-threaded version is slower than it should be, thus boosting the calculated gain to be larger than it can be. To solve this problem one would have to make sure that the same algorithm in both programs takes the same time to handle 1 byte of data at its lowest level. Only then is it possible to compare both programs and precisely determine the performance advantage of multithreading.

[1] see §3.6.1 for how the gain is calculated

5 Conclusion

Integrity protection is a fundamental security mechanism for 5G to ensure data integrity, authenticity, and confidentiality. Despite these crucial benefits for the security of 5G, integrity protection is *optional* to use. Performance impairments like a higher latency or increased power consumption contradict the high intended performance standards of 5G and are an obstacle to the large-scale use of integrity protection. In this work, we provided a first analysis of the potential overhead of integrity protection algorithms and assessed potential algorithmic and conceptual improvements. We found that the current choice of specified algorithms introduces unnecessary throughput and latency impairments. At the same time, alternative algorithms like Poly1305, and slight conceptual adjustments like parallelization and hardware optimization enable significant performance gains. With our findings, we support the change from optional to desirable UP IP.

5.1 Future Work

Future work should analyze suitable candidate algorithms on specialized hardware and different architectures. Furthermore, the evaluation should be extended to other performance criteria like battery drainage and power consumption. Also, it would be interesting to see the theoretical performance and power boundaries on different architectures as well as the price per bit of data sent in terms of CPU cycles. This in turn would be helpful to compare the results of general-purpose CPUs with dedicated hardware, such as Field Programmable Gate Array (FPGA) or Application-specific integrated circuit (ASIC) based chips.

For the future of 5G, it is also interesting to investigate the impact of longer symmetric key lengths, considering the discussion of going from 128-bit to 256-bit key lengths for Post-Quantum Cryptography (PQC).

For realistic insights on the overhead of UP IP, only experiments on suitable candidate devices provide representative performance characteristics. Such phones will become available with release 16, which is expected mid-2022.

A Acronyms

UP IP User Plane Integrity Protection

SA Standalone

SIMD Single Instruction Multiple Data

CMAC Block cipher-based MAC

C-RAN Cloud Radio Access Network

3GPP 3rd Generation Partnership Project

AES Advanced Encryption Standard

AEAD Authenticated Encryption with Associated Data

ASIC Application-specific integrated circuit

CK Confidentiality Key

DNS Domain Name System

DNS Domain Name System

EEA Enhanced Encryption Algorithm

EIA Enhanced Integrity Algorithm

FPGA Field Programmable Gate Array

GCM Galois/Counter Mode

gNodeB Next Generation NodeB

IK Integrity Key

IP Internet Protocol

LTE Long Term Evolution

MAC Message Authentication Code

NIST National Institute of Standards and Technology

PDCP Packet Data Convergence Protocol

PQC Post-Quantum Cryptography

RAN Radio Access Network

SAGE Security Algorithms Group of Experts

SCA Side Channel Analysis

UE User Equipment

VoLTE Voice over LTE

List of Figures

2.1 RAN Architecture in 5G 7
2.2 C-RAN Architecture in 5G 8
2.3 MAC Principle 10
2.4 Inputs and Output of EIAx 12
2.5 Structure of EIA1's MAC Calculation 13
2.6 Structure of EIA2's MAC Calculation 14
2.7 Inputs and Output of EEAx 16

3.1 System Concept of Influencing Factors. 17
3.2 Key Stream Generation vs. MAC Calculation vs. Memory Usage . . 19
3.3 Throughput of Existing and Alternative Integrity Protection Algorithms 21
3.4 Throughput of Encryption Algorithms 22
3.5 Schematic Representation of the Parallel Execution for TX Direction. 24
3.6 Performance Gain of our Multi-Threaded Integrity Computation over Single-Threaded variant. 25

List of Tables

3.1 Absolute Throughput of EIAx and EEAx for 1465 bytes input 23
3.2 Vendors and ISA Support for Crypto Algorithms 28
3.3 Summary of Findings 29

Bibliography

[1] www.3gpp.org/ftp/tsg_sa/TSG_SA/TSGS_88E_Electronic/Docs/SP-200537.zip.

[2] Active Attack: aLTEr. https://alter-attack.net/.

[3] 5g-security. https://www.ericsson.com/en/reports-and-papers/white-papers/5g-security---enabling-a-trustworthy-5g-system.

[4] IMP4GT Attacks. https://imp4gt-attacks.net/.

[5] 3gpp_tsg_ran@list.etsi.org archive, subject: Re: [integrity_protection] final decision on rp-200505, 2020.

[6] 3GPP. 3gpp. ts 23.501 (v17.1.1): 3rd generation partnership project; technical specification group services and system as- pects; system architecture for the 5G system (5GS). June 2021. https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3144.

[7] 3GPP. 3GPP System Architecture Evolution (SAE); Security architecture. Technical Specification (TS) 33.401, 3rd Generation Partnership Project (3GPP), 12 2021. URL http://www.3gpp.org/DynaReport/33401.htm. Version 17.0.0.

[8] 3GPP. NR; Packet Data Convergence Protocol (PDCP) specification. Technical Specification (TS) 38.323, 3rd Generation Partnership Project (3GPP), 12 2021. URL http://www.3gpp.org/DynaReport/38323.htm. Version 16.6.0.

[9] 3GPP. Security architecture and procedures for 5G System. Technical Specification (TS) 33.501, 3rd Generation Partnership Project (3GPP), 1 2022. URL http://www.3gpp.org/DynaReport/33501.htm. Version 17.4.1.

[10] N. Ahmad, R. Hasan, and W. M. Jubadi. Design of aes s-box using combinational logic optimization. In *2010 IEEE Symposium on Industrial Electronics and Applications (ISIEA)*, pages 696–699. IEEE, 2010.

[11] ARM. Arm architecture reference manual armv8, for a-profile architecture. https://developer.arm.com/documentation/ddi0487/latest/.

[12] E. Barkan, E. Biham, and N. Keller. Instant ciphertext-only cryptanalysis of gsm encrypted communication. In *Annual international cryptology conference*, pages 600–616. Springer, 2003.

[13] C. Beierle, P. Derbez, G. Leander, G. Leurent, H. Raddum, Y. Rotella, D. Rupprecht, and L. Stennes. Cryptanalysis of the gprs encryption algorithms gea-1 and gea-2. Cryptology ePrint Archive, Report 2021/819, 2021. https://ia.cr/2021/819.

[14] M. Bellare and C. Namprempre. Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm. *Journal of Cryptology*, 21:469–491, 2008.

[15] D. J. Bernstein. A state-of-the-art message-authentication code. https://cr.yp.to/mac.html.

[16] E. Biham, O. Dunkelman, and N. Keller. A related-key rectangle attack on the full kasumi. In B. Roy, editor, *Advances in Cryptology - ASIACRYPT 2005*, pages 443–461, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg. ISBN 978-3-540-32267-2.

[17] G. Brown. Cloud RAN & the Next-Generation Mobile Network Architecture. 2017.

[18] M. J. Dworkin. *Sp 800-38d. recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac.* National Institute of Standards & Technology, 2007.

[19] M. J. Dworkin et al. Recommendation for block cipher modes of operation: The cmac mode for authentication. 2016.

[20] P. Ekdahl, T. Johansson, A. Maximov, and J. Yang. A new snow stream cipher called snow-v. *Transactions on Symmetric Cryptology.*

[21] P. Ekdahl, A. Maximov, T. Johansson, and J. Yang. SNOW-Vi: An Extreme Performance Variant of SNOW-V for Lower Grade CPUs. In *Proceedings of the 14th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, WiSec '21, page 261–272, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383493. doi: 10.1145/3448300.3467829. URL https://doi.org/10.1145/3448300.3467829.

[22] V. G. Erdinc Ozturk. Enabling high-performance galois-counter-mode on intel architecture processors. https://www.intel.cn/content/dam/www/public/us/en/documents/software-support/enabling-high-performance-gcm.pdf.

[23] Ericsson. 5G Core (5GC). https://www.ericsson.com/en/core-network/5g-core.

[24] ETSI. ETSI TS 133 401 V15.7.0. https://www.etsi.org/deliver/etsi_ts/133400_133499/133401/15.07.00_60/ts_133401v150700p.pdf.

[25] ETSI/SAGE. Specification of the 3gpp confidentiality and integrity algorithms uea2 uia2. document 1: Uea2 and uia2 specification. https://www.gsma.com/security/wp-content/uploads/2019/05/uea2uia2d1v21.pdf.

[26] E. H. Ghizlane Orhanou. The new lte cryptographic algorithms eea3 and eia3, . http://www.naturalspublishing.com/files/published/zf215q192wiz2s.pdf.

[27] E. H. Ghizlane Orhanou. Utran cryptographic algorithms operation and complesity study, . http://www.jatit.org/volumes/research-papers/Vol14No2/4Vol14No2.pdf.

[28] I. Gomez-Miguelez, A. Garcia-Saavedra, P. D. Sutton, P. Serrano, C. Cano, and D. J. Leith. srsLTE: an open-source platform for LTE evolution and experimentation. In *ACM International Workshop on Wireless Network Testbeds, Experimental Evaluation and Characterization (WiNTECH)*, 2016.

[29] M. M. Grant Hernandez. Reversing & emulating samsung's shannon baseband. https://hardwear.io/netherlands-2020/presentation/samsung-baseband-hardwear-io-nl-2020.pdf.

[30] S. Gueron. Intel advanced encryption standard (aes) new instructions set. https://www.intel.com/content/dam/doc/white-paper/advanced-encryption-standard-new-instructions-set-paper.pdf.

[31] D. Harold and J. BernierSantarini. Mediatek selects mips for lte modems. https://www.mips.com/press/mediatek-selects-mips-for-lte-modems/.

[32] R. Housley. Using ChaCha20-Poly1305 Authenticated Encryption in the Cryptographic Message Syntax (CMS). *RFC*, 8103:1–9, 2017.

[33] ITU. Detailed specifications of the terrestrial radio interfaces of international mobile telecommunications-2020 m.2150-0, 2021.

[34] John M Meredith. Study on the support of 256-bit algorithms for 5G. https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3422.

[35] P. Kitsos, N. Sklavos, G. Provelengios, and A. N. Skodras. Fpga-based performance analysis of stream ciphers zuc, snow3g, grain v1, mickey v2, trivium and e0. *Microprocessors and Microsystems*.

[36] P. Kitsos, G. Selimis, and O. Koufopavlou. High performance ASIC implementation of the SNOW 3G Stream Cipher. In *Conference on Very Large Scale Integration (VLSI-SOC '08)*, 2008.

[37] H. Kuo and I. Verbauwhede. Architectural optimization for a 1.82 gbits/sec vlsi implementation of the aes rijndael algorithm. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 51–64. Springer, 2001.

[38] N. Mavrogiannopoulos, J. Strombergson, A. Langley, W.-T. Chang, and S. Josefsson. The ChaCha Stream Cipher for Transport Layer Security. 2015.

[39] N. Méloni, C. Nègre, and M. A. Hasan. High Performance GHASH Function for Long Messages. In *ACNS*, 2010.

[40] X. mi Zhao and S. Guo. High-performance hardware implementation of the 3gpp algorithm kasumi. *The Journal of China Universities of Posts and Telecommunications*, 13:60–62, 2006.

[41] MIPS. The mips dsp application-specific extension to the mips32 architecture v3.01. `https://www.mips.com/products/architectures/ase/dsp/`.

[42] Y. Nir and A. Langley. ChaCha20 and Poly1305 for IETF Protocols. *RFC*, 8439, 2018.

[43] C. Paar and J. Pelzl. Message Authentication Codes (MACs). 2010.

[44] M. Polubelova, K. Bhargavan, J. Protzenko, B. Beurdouche, A. Fromherz, N. Kulatova, and S. Zanella-Béguelin. Haclxn: Verified generic simd crypto (for all your favourite platforms). In *Conference on Computer and Communications Security (CCS)*, 2020.

[45] L. Prasad. An energy efficient radio base station. 2018.

[46] Qualcomm. Hexagon v66 hvx, programmer's reference manual, 80-n2040-44 rev. b. `https://developer.qualcomm.com/software/hexagon-dsp-sdk`.

[47] RedHead. Virtualize radio access networks for more agility and choice. `https://www.redhat.com/rhdc/managed-files/ve-telco-vran-solution-brief-f25885wg-202011-en.pdf`.

[48] D. Rupprecht, K. Kohls, T. Holz, and C. Pöpper. Breaking LTE on Layer Two. In *IEEE Symposium on Security & Privacy (SP)*. IEEE, 2019.

[49] D. Rupprecht, K. Kohls, T. Holz, and C. Pöpper. Imp4gt: Impersonation attacks in 4g networks. In *ISOC Network and Distributed System Security Symposium (NDSS)*. ISOC, Feb. 2020.

[50] H. Seo, G. Lee, T. Park, and H. Kim. Compact gcm implementations on 32-bit armv7-a processors. In *Conference on Information and Communication Technology Convergence*, 2017.

[51] S. Traboulsi, M. Sbeiti, F. Bruns, S. Hessel, and A. Bilgic. An optimized parallel and energy-efficient implementation of snow 3g for lte mobile devices. In *2010 IEEE 12th International Conference on Communication Technology*, pages 535–538, 2010. doi: 10.1109/ICCT.2010.5688900.

[52] D. Yamamoto, K. Itoh, and J. Yajima. A very compact hardware implementation of the kasumi block cipher. In *Information Security Theory and Practices*, 2010.

[53] J. Yang. *Contributions to Confidentiality and Integrity Algorithms for 5G*. PhD thesis, Department of Electrical and Information Technology, Nov. 2021. Defence details Date: 2021-12-17 Time: 09:00 Place: Lecture hall E:A, building E, John Ericssons väg 2, Faculty of Engineering LTH, Lund University, Lund. External reviewer(s) Name: Cid, Carlos Title: Prof. Affiliation: University of London, United Kingdom. —.